TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN – ĐHQG-HCM
KHOA KỸ THUẬT MÁY TÍNH

KIẾN TRÚC MÁY TÍNH

LAB 01

LÀM QUEN MARS: XỬ LÝ I/O và CẤU TRÚC ĐIỀU KIỆN

Họ và tên	Dương Gia Thịnh	**MSSV**	25521759
Lớp	VMTN2025	**Ngày thực hành**	10/06/2026
Tự chấm	10	**Link git**	https://github.com/TDuong512

1. Tổng quan buổi thực hành

Sinh viên dùng trình mô phỏng **MARS 4.5** để biến kiến thức Chương 2 (kiến trúc bộ lệnh MIPS: toán hạng, số có dấu/không dấu, biểu diễn lệnh, lệnh điều kiện và nhảy) thành các thí nghiệm dự đoán – lập trình – quan sát – giải thích. Tình huống xuyên suốt: một chương trình tính tiền điện sinh hoạt theo biểu giá bậc thang, kèm một đoạn giai điệu ngắn phát bằng syscall MIDI để báo hoàn tất.

Sau buổi thực hành, Sinh viên có thể:

- Soạn, Assemble, chạy từng bước và gỡ lỗi chương trình MIPS32 trên MARS; dùng syscall nhập/xuất, thời gian hệ thống và MIDI.
- Phân biệt dòng lệnh nguồn, giả lệnh và lệnh cơ bản; đọc cột Code/Basic của Text Segment và tự giải mã các trường của lệnh dạng R, dạng I và độ lệch của lệnh nhánh.
- Hiện thực cấu trúc điều kiện và vòng lặp bằng `slt`/`sltu`/`beq`/`bne`; truy cập mảng trong bộ nhớ bằng địa chỉ cơ sở và chỉ số.
- Phân biệt so sánh có dấu và không dấu khi kiểm tra miền giá trị của dữ liệu nhập.
- Dùng Instruction Counter để đo số lệnh thực thi, xây dựng mô hình chi phí theo dữ liệu vào và đánh giá một thay đổi thiết kế bằng số liệu đo.

Phần	Nội dung	Thời gian gợi ý	Điểm
Phần 0	Làm quen MARS, kiểm tra môi trường	15 phút	
TN 1	Từ lệnh nguồn đến mã máy: giả lệnh, Text Segment, Instruction Counter	20 phút	1,5
TN 2	Tính tiền điện bậc thang: nhập có kiểm tra, biểu giá lưu trong bộ nhớ	70 phút	4,0
TN 3	Phát giai điệu bằng syscall MIDI và đo thời gian chạy	25 phút	2,0
TN 4	Đo số lệnh và giải mã lệnh trên chương trình của mình	30 phút	2,0
Kết	Tổng kết, phản tư, nộp bài	20 phút	0,5
Tổng		**180 phút**	**10,0/10,0**

Quy ước làm bài

- **Ô nền vàng nhạt** là chỗ sinh viên điền hoặc trả lời trực tiếp vào file này (gõ chữ, hoặc dán ảnh chụp màn hình vào khung nét đứt).
- **Tham số cá nhân.** `a` là chữ số cuối của MSSV, `b` là chữ số liền trước. Mọi số liệu riêng của bài (đơn giá, nhịp độ, nhạc cụ, giá trị kiểm thử) được tính từ `a` và `b`; điền vào Bảng 1 trước khi bắt đầu.
- **Quy ước mã nguồn.** Mỗi chương trình có `.data`, `.text`, nhãn `main` và kết thúc bằng syscall 10. Chuỗi in ra và chú thích dùng ASCII không dấu. Ghi chú vai trò thanh ghi theo quy ước: `$t*` giá trị tạm, `$s*` giá trị cần giữ qua syscall, `$v0` số hiệu syscall, `$a0`–`$a3` đối số.
- Mỗi thí nghiệm theo chu trình **DỰ ĐOÁN → QUAN SÁT → GIẢI THÍCH**. Ghi dự đoán **trước** khi chạy trên MARS; dự đoán sai **không bị trừ điểm**, phần giải thích vì sao sai mới được chấm.

PHẦN 0 — Làm quen MARS (15 phút)

Giảng viên hướng dẫn nhanh giao diện MARS. Sinh viên tự đánh dấu ☑ khi thực hiện được từng kỹ năng dưới đây (bỏ qua nếu đã thành thạo).

☑	Kỹ năng	Gợi ý
☑	Tạo file mới, lưu với tên `Lab01_<MSSV>_dien.asm`	Mỗi chương trình nộp là một file riêng; xem quy ước ở cuối tài liệu.
☑	Assemble (Run ▸ Assemble, F3), chạy (Run ▸ Go, F5), chạy từng lệnh (F7), lùi một lệnh (F8), đặt lại (F12)	Sau khi sửa mã phải Assemble lại. Lỗi Assemble hiện ở tab Mars Messages.
☑	Kiểm tra Settings: bỏ chọn **Delayed branching**; chọn **Permit extended (pseudo) instructions and formats**	Hai thiết lập này ảnh hưởng trực tiếp đến mã máy và số lệnh đếm được.
☑	Đọc các vùng của tab Execute: Text Segment (Address, Code, Basic, Source), Data Segment, Registers	Tích **Labels** ở dưới Data Segment để xem địa chỉ của nhãn dữ liệu.
☑	Đặt breakpoint (ô Bkpt) và chạy từng bước để quan sát thanh ghi đổi màu	Thanh ghi vừa bị ghi sẽ được tô sáng.
☑	Mở Tools ▸ Instruction Counter và bấm **Connect to MIPS** trước khi chạy	Công cụ chỉ đếm khi đã kết nối; bấm **Reset** để đếm lại từ 0.
☑	Tra bảng syscall trong Help ▸ Help (thẻ MIPS, mục Syscalls)	Bảng tóm tắt các syscall dùng trong buổi này nằm ngay dưới.
☑	Chụp màn hình cửa sổ MARS và dán vào khung ảnh của file này	Chụp cả vùng Run I/O lẫn vùng mã đang xét.

> **Các syscall và lệnh dùng trong buổi này**
> Đặt số hiệu dịch vụ vào `$v0`, đối số vào `$a0`–`$a3`, rồi thực hiện `syscall`.

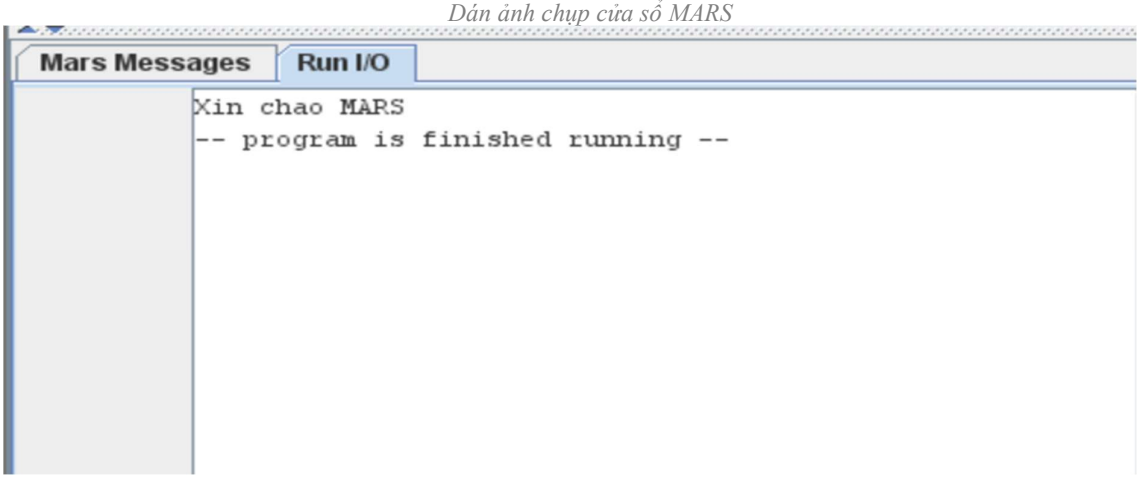
$v0	Dịch vụ	Đối số	Kết quả
1	In số nguyên	`$a0` = giá trị	
4	In chuỗi kết thúc bằng 0	`$a0` = địa chỉ chuỗi	
5	Đọc số nguyên có dấu		`$v0` = giá trị đọc được
10	Kết thúc chương trình		
30	Lấy thời gian hệ thống (ms)		`$a0` = 32 bit thấp, `$a1` = 32 bit cao
31	Phát một nốt MIDI, trả về ngay	`$a0` cao độ (0–127), `$a1` độ dài (ms), `$a2` nhạc cụ (0–127), `$a3` âm lượng (0–127)	
32	Chờ (ngủ)	`$a0` = số ms	
33	Phát một nốt MIDI, chờ đến khi nốt kết thúc	như syscall 31	

Lệnh `mul rd, rs, rt` nhân hai số và ghi 32 bit thấp của tích vào `rd` (cơ chế bên trong sẽ học ở Chương 3, ở đây dùng như một lệnh có sẵn).

Mục	Ghi nhận
Phiên bản MARS đang dùng (Help ▸ About)	MARS 4.5
Máy có phát được âm thanh qua MIDI không? (Có / Không)	Có

Bước 0 – Kiểm tra nhanh. Tạo chương trình in dòng chữ `Xin chao MARS` rồi kết thúc (chỉ dùng syscall 4 và 10). Assemble, chạy và dán ảnh chụp vùng Run I/O vào khung dưới. File này chỉ để làm quen, không nộp.

Dán ảnh chụp cửa sổ MARS



sau khi chạy chương trình kiểm tra (thấy rõ dòng in ra)

Tham số cá nhân

Điền `a`, `b` rồi tính các giá trị còn lại. Các giá trị này dùng cho TN 2 và TN 3.

Bảng 1. Tham số cá nhân

Đại lượng	Cách tính	Giá trị của Sinh viên
a	Chữ số cuối của MSSV	9
b	Chữ số liền trước chữ số cuối của MSSV	5
Đơn giá bậc 1 (đồng/kWh)	1600 + 20*a	1780
Số nửa cung dịch giọng	a	9
Độ dài một đơn vị nhịp (ms)	150 + 10*b	200
Nhạc cụ (số hiệu chương trình MIDI)	Phần dư của b khi chia 4 là 0, 1, 2, 3 tương ứng 0, 24, 40, 73	24
Số kWh kiểm thử cá nhân x_b	100 + 25*b	225

THÍ NGHIỆM 1 — Từ lệnh nguồn đến mã máy

Thời lượng gợi ý: 20 phút · File: tn1.asm (không nộp) · Điểm: 1,5

Tình huống

Trước khi viết chương trình lớn, kỹ sư cần biết một dòng mã nguồn thực sự trở thành bao nhiêu lệnh máy: số lệnh này quyết định kích thước chương trình và (cùng với số lệnh thực thi) thời gian chạy. Thí nghiệm này khảo sát một chương trình nhỏ để làm rõ ba con số: số dòng nguồn, số lệnh cơ bản trong bộ nhớ và số lệnh thực sự được thực thi.

Chương trình khảo sát tn1.asm (gõ lại nguyên văn, chưa Assemble)

```
        .data
msg:    .asciiz "Ket qua: "
        .text
main:   li   $t0, 25
        li   $t1, 70000
        add  $t2, $t0, $t1
        blt  $t0, $t1, small
        li   $t2, 0
small:  la   $a0, msg
        li   $v0, 4
        syscall
        move $a0, $t2
        li   $v0, 1
        syscall
        li   $v0, 10
        syscall
```

Bước 1.1 – Dự đoán trước khi Assemble. Chưa chạy MARS. Đọc chương trình và điền cột Dự đoán của Bảng 2 và Bảng 3.

DỰ ĐOÁN Mỗi dòng nguồn ở Bảng 2 sinh ra bao nhiêu lệnh cơ bản? Hai lệnh trong Bảng 3 là bao nhiêu dòng nguồn, lệnh cơ bản (trong bộ nhớ) và lệnh được thực thi? Chương trình in ra gì?

Bước 1.2 – Assemble và quan sát. Assemble, đọc Text Segment, điền cột Quan sát. Mở Instruction Counter (Connect to MIPS), chạy chương trình và đọc tổng số lệnh.

QUAN SÁT Cột Basic cho biết các lệnh cơ bản thực sự nằm trong bộ nhớ; cột Source là dòng nguồn tương ứng.

Bảng 2. Số lệnh cơ bản sinh ra từ mỗi dòng nguồn

Dòng nguồn	Dự đoán (số lệnh)	Quan sát (số lệnh)	Các lệnh cơ bản quan sát được (cột Basic)
`li $t0, 25`	1	1	addiu $8,$0,0x00000019
`li $t1, 70000`	2	2	lui $1,0x00000001 ori $9,$1,0x00001170
`blt $t0, $t1, small`	2	2	slt $1,$8,$9 bne $1,$0,0x00000001
`la $a0, msg`	2	2	lui $1,0x0001001 ori $4,$1,0x00000000
`move $a0, $t2`	1	1	addu $4,$0,$10

Bảng 3. Số dòng lệnh nguồn, lệnh cơ bản, lệnh được thực thi của chương trình khảo sát

Đại lượng	Dự đoán	Quan sát	Nguồn đọc kết quả
Số dòng lệnh nguồn trong `.text`	13	13	Đếm trong Source
Số lệnh cơ bản trong bộ nhớ (tĩnh)	16	16	Số hàng của Text Segment
Số lệnh được thực thi (động)	15	15	Instruction Counter, mục Total
Chuỗi in ra Run I/O	Ket qua: 70025	Ket qua: 70025	Cửa sổ Run I/O

Câu 1.1 (Hiểu). Vì sao `li $t0, 25` chỉ cần một lệnh cơ bản còn `li $t1, 70000` cần hai? Giá trị 70000 không vừa trường nào của lệnh dạng I? Thanh ghi `$at` (số 1) xuất hiện ở đâu trong Text Segment và đóng vai trò gì?

Trả lời: Vì trong lệnh I gồm 4 trường mà trường immediate chỉ có 16 bit nên chỉ có thể biểu diễn tối đa là 2^16 = 65536 đối với bit không dấu , số 70000 lớn hơn số có thể biểu diễn do đó nó cần tách thành 2 lệnh.
- Vị trí xuất hiện: Nằm ở lệnh lui $at, ... (thanh ghi số 1) khi mã máy giải mã lệnh li $t1, 70000.
Vai trò: Là thanh ghi tạm dành riêng cho trình hợp dịch dùng làm biến trung gian để lưu 16 bit cao trước khi ghép giá trị hoàn chỉnh vào thanh ghi đích, tránh làm thay đổi dữ liệu của người dùng.

Câu 1.2 (Phân tích). Giải thích vì sao ba con số ở Bảng 3 khác nhau. Trong hai lệnh `blt` và `li $t2, 0`, lệnh nào góp phần làm số lệnh thực thi nhỏ hơn số lệnh trong bộ nhớ và vì sao?

Trả lời: 3 con số khác nhau do giả lệnh nguồn khi chạy có thể được dịch ra thành nhiều lệnh khác nhau , đồng thời số lệnh thực thi còn phụ thuộc vào việc có nhảy nhánh hay không . Lệnh góp phần làm số lệnh thực thi nhỏ hơn số lệnh trong bộ nhớ là blt vì khi câu lênh này đúng thì nó sẽ nhảy tới nhánh small , lúc này chương trình sẽ bỏ qua lệnh l $t2, 0

Bước 1.3 – Giải mã bằng tay. Đọc mã hex của lệnh `add $t2, $t0, $t1` ở cột Code **trước**, tách các trường rồi so với cột Basic. Làm tương tự với lệnh nhánh do `blt` sinh ra.

Bảng 4. Giải mã lệnh dạng R: `add $t2, $t0, $t1`

Địa chỉ	Mã hex (Code)	opcode	rs	rt	rd	shamt	funct
0x0040000c	0x01095020	000000	01000	01001	01010	00000	0x20

Bảng 5. Giải mã lệnh nhánh sinh ra từ `blt` (lệnh thứ hai của cặp)

Bne $1 $8 $9

Địa chỉ	Mã hex (Code)	opcode	rs	rt	immediate (thập phân, có dấu)	Địa chỉ nhãn `small`
0x00400014	0x14200001	000101	00001	00000	00000001	0x0040001c

Câu 1.3 (Áp dụng). Viết phép tính cho thấy trường immediate của lệnh nhánh ở Bảng 5 được suy ra từ địa chỉ của nó và địa chỉ nhãn `small` (nêu rõ vì sao có số hạng 4 và phép chia 4). Nếu chèn thêm 3 lệnh cơ bản vào giữa `blt` và `small`, trường immediate đổi thành bao nhiêu?

Trả lời: Phép tính: immediate = (0x0040001c - (0x00400014 + 4)) / 4 = 1.
Giải thích : có số hạng 4 vì một lần thanh ghi PC nhảy lệnh có độ dài là 4 byte (1 word) , có phép chia 4 là vì mỗi lệnh là 1 word tương đương 4 byte nên khi các lệnh được lưu trữ trong bộ nhớ sẽ cách nhau 4 đơn vị khi ta thực hiện phép chia 4 sẽ biết được số lệnh cách nhau
- *Nếu chèn thêm 3 lệnh cơ bản , thì trường immmediate sẽ tăng lên 4 do 1 + 3 = 4*

Bước 1.4 – Dự đoán – quan sát lần hai. Trong tn1.asm thay `blt` bằng `bgt` (giữ nguyên nhãn và các dòng còn lại).

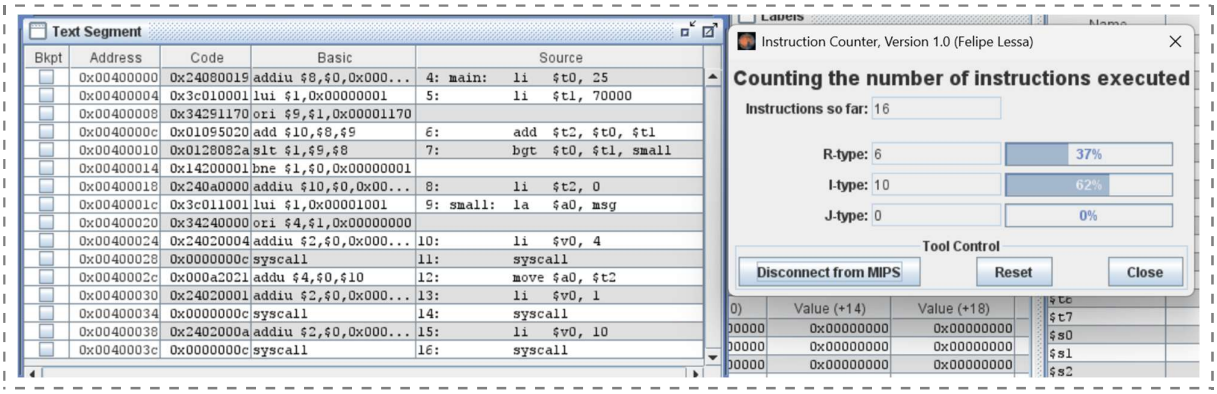
DỰ ĐOÁN Số lệnh thực thi bây giờ là bao nhiêu và chuỗi in ra là gì? Ghi dự đoán vào Bảng 6, Assemble, đặt lại Instruction Counter, chạy và ghi quan sát.

Bảng 6. Thay `blt` bằng `bgt`

Đại lượng	Dự đoán	Quan sát
Số lệnh được thực thi	16	16
Chuỗi in ra	Ket qua: 0	Ket qua: 0

Câu 1.4 (Phân tích). Ba lệnh `syscall` trong chương trình có cùng mã máy hay khác nhau (ghi mã hex quan sát được)? Nếu giống nhau thì máy phân biệt dịch vụ cần thực hiện bằng cách nào?

Trả lời: ba lệnh syscall có cùng mã máy (0x0000000c). Máy phân biệt được là nhờ giá trị được lưu vào $v0 ngay trước lệnh syscall được gọi



THÍ NGHIỆM 2 — Tính tiền điện bậc thang

Thời lượng gợi ý: 70 phút · File nộp: Lab01_<MSSV>_dien.asm · Điểm: 4,0

Tình huống

Một công ty điện lực cần chương trình tính tiền điện sinh hoạt của một hộ trong một tháng. Điện năng tiêu thụ được chia thành sáu bậc; **mỗi kWh được tính theo đơn giá của bậc chứa nó**, và tổng tiền là tổng của các phần ở từng bậc. Biểu giá dưới đây là biểu giá giả lập dùng cho bài thực hành. Chương trình chạy trên một vi xử lý nhúng nhỏ nên bảng giá phải nằm trong bộ nhớ dữ liệu để thay đổi khi biểu giá đổi mà không phải sửa phần lệnh.

Bảng 7. Biểu giá giả lập (đồng/kWh, số nguyên)

Bậc	Từ (kWh)	Đến (kWh)	Đơn giá (đồng/kWh)
1	0	50	1600 + 20*a
2	51	100	1660 + 20*a
3	101	200	1950 + 20*a
4	201	300	2500 + 20*a
5	301	400	2850 + 20*a
6	401	9999	3000 + 20*a

Đặc tả chương trình

- **Nhập.** In lời nhắc `Nhap so kWh (0..9999): ` rồi đọc một số nguyên có dấu bằng syscall 5.
- **Kiểm tra.** Giá trị hợp lệ khi $0 \le x \le 9999$. Nếu không hợp lệ, in `Gia tri khong hop le, nhap lai.` rồi nhắc nhập lại cho đến khi hợp lệ. Việc kiểm tra miền giá trị chỉ được dùng **một** lệnh so sánh và **một** lệnh nhánh.
- **Dữ liệu.** Ngưỡng các bậc và đơn giá lưu trong `.data` dưới dạng mảng `.word`. Phần tính toán duyệt các mảng này bằng vòng lặp; đổi giá trị trong mảng thì không phải sửa phần lệnh. Không viết riêng mã cho từng bậc.
- **Xuất.** In đúng khuôn `Tien dien: <so tien> dong` rồi xuống dòng, sau đó kết thúc chương trình.
- **Giao diện thanh ghi.** Ghi chú ở đầu file vai trò của từng thanh ghi `$s*` đã dùng.

Bước 2.1 – Dự đoán bằng tay. Chưa viết mã. Điền đơn giá 6 bậc của Sinh viên (từ Bảng 1) vào Bảng 8, rồi tính tay tiền điện cho các giá trị x ở Bảng 10 (có thể dùng máy tính bỏ túi).

DỰ ĐOÁN Ghi tiền điện dự kiến cho từng x. Chú ý các giá trị nằm đúng trên hoặc sát ranh giới bậc.

Bảng 8. Đơn giá 6 bậc theo tham số a của Sinh viên

Bậc	1	2	3	4	5	6
Đơn giá	1780	1840	2130	2680	3030	3180

Bước 2.2 – Thiết kế trước khi viết mã. Mô tả thuật toán bằng lời hoặc mã giả (không cần đúng cú pháp) và lập bảng phân công thanh ghi.

THIẾT KẾ Ghi rõ điều kiện kết thúc vòng lặp, cách xác định phần điện năng thuộc mỗi bậc và vì sao chương trình không cần biết trước số bậc đã dùng.

Thuật toán: trước tiên khai báo các lệnh in ra màn hình sẽ gọi sắp tới và các bậc, đơn giá trong ,data
Sau đó: đầu tiên cho nhập vào giá trị x (kWh) rồi kiểm tra điều kiện 0 <= x <= 9999 bằng phép so sánh và sẽ nhảy báo lỗi rồi bắt nhập lại .Sau đó khởi tạo 2 thanh ghi để lưu tổng tiền và số kWh còn lại.
Thuật toán vòng lặp :
+ Điều kiện kết thúc kWh_left <= 0 (Do đó chương trình không cần biết trước số bậc đã dùng)
*+ Cách xác định phần điện năng thuộc mỗi bậc : tại bậc hiện tại ta so sánh số kWh còn lại so với số kWh trong từng khung bậc . Nếu số kWh còn lại >= dung lượng bậc thì ta sẽ tính là dung lượng bậc x đơn giá rồi lấy kWh còn lại trừ dung lượng bậc . Nếu số kWh_left <= dung lượng bậc thì ta tính kWh_left * đơn giá , gán số kWh_left về 0 rồi thoát vòng lặp*
+ Tại mỗi lần tính toán ta sẽ tiến hành cộng dồn vào tổng tiền và tăng con trỏ để có thể xét tiếp bậc tiếp theo

Bảng 9. Phân công thanh ghi

Thanh ghi	Vai trò trong chương trình
$s0	Lưu số kWh nhập vào (x).
$s1	Lưu tổng số tiền điện
$s2	Con trỏ duyệt mảng dung lượng bậc
$s3	Con trỏ duyệt mảng đơn giá
$t0	Dùng để kiểm tra điều kiện của x
$t1	Lưu giá trị của dung lượng tại bậc hiện tại
$t2	Lưu giá trị của giá tiền tại bậc hiện tại
$t3	Lưu giá trị (tiền điện * dung lượng) hoạc (số kWh còn lại * giá tiền) tại bậc hiện tại
$t4	Số kWh còn lại

Bước 2.3 – Lập trình. Viết chương trình theo đặc tả trong file `Lab01_<MSSV>_dien.asm`. Assemble, sửa lỗi, chạy từng bước ít nhất một lần để theo dõi vòng lặp qua các bậc.

LẬP TRÌNH Khi gỡ lỗi, đặt breakpoint ở đầu thân vòng lặp và theo dõi các thanh ghi tổng tiền, cận dưới và con trỏ vào mảng.

Bước 2.4 – Kiểm thử và so sánh với dự đoán. Chạy chương trình lần lượt với các giá trị ở Bảng 10 và Bảng 11.

QUAN SÁT Ghi kết quả chương trình in ra và đối chiếu với dự đoán. Nếu lệch, tìm xem dự đoán tay hay chương trình sai, ghi kết luận vào cột cuối.

Bảng 10. Tiền điện: dự đoán và quan sát

x (kWh)	Dự đoán (đồng)	Quan sát (đồng)	Khớp? Nếu lệch, nguyên nhân
30	53400	53400	Khớp
50	89000	89000	Khớp
51	90840	90840	Khớp
120	223600	223600	Khớp
250	528000	528000	Khớp
401	968180	968180	Khớp
x_b = 225	461000	461000	Khớp

Bảng 11. Dữ liệu nhập không hợp lệ và giá trị biên của miền nhập

Dữ liệu nhập	Dự đoán hành vi	Quan sát
0	In : Tien dien: 0 dong	Tien dien: 0 dong
9999	In ra số tiền điện	Tien dien: 31489820 dong
10000	Báo lỗi và bắt nhập lại	Gia tri khong hop le, nhap lai.
−1	Báo lỗi và bắt nhập lại	Gia tri khong hop le, nhap lai.

Câu 2.1 (Phân tích). Chỉ rõ lệnh so sánh và lệnh nhánh dùng để kiểm tra miền [0, 9999] trong chương trình của Sinh viên. Giải thích vì sao **một** phép so sánh bắt được cả $x < 0$ lẫn $x \geq 10000$.

Trả lời: Trong chương trình em sử dụng một lệnh so sánh là sltiu $t0, $s0, 10000 nghĩa là nếu giá trị tại thanh ghi s0 nhỏ hơn 10000 (tức là x <=9999) thì thanh ghi $t0 = 1 , nếu nhập x <0 tức là số âm thì giá trị biểu diễn bù 2 sẽ chuyển thành số không dấu rất lớn . Sau đó em dùng lệnh beq $t0, $zero, wronginput để so sánh xem t0 = 0 ? hay không , nếu = 0 tức là điều kiện sai và sẽ nhảy tới nhánh wronginput để nhập lại

Bước 2.5 – Thí nghiệm có dấu và không dấu. Đổi lệnh so sánh ở câu 2.1 sang phiên bản so sánh **ngược** kiểu (có dấu thành không dấu hoặc ngược lại), Assemble lại, nhập $x = -1$.

DỰ ĐOÁN Chương trình sẽ làm gì? Ghi dự đoán trước, rồi chạy và ghi quan sát. Sau khi xong, khôi phục lệnh đúng.

Dự đoán	Quan sát
Chương trình hiểu $-1 < 10000$ là đúng, không báo lỗi mà chuyển sang tính toán và in kết quả	Tien dien: 0 dong

Câu 2.2 (Phân tích). Giải thích kết quả quan sát bằng cách xét biểu diễn bù hai của −1 và cách hai lệnh so sánh diễn giải cùng một chuỗi bit.

Trả lời:số -1 trong máy được biểu diễn dưới dạng một chuỗi bit 1 , Nên khi dùng lệnh slti máy sẽ thấy bit MSB = 1 tức là số này đang âm nên mặc định -1 < 10000 là đúng còn khi dùng sltiu (so sánh không dấu) thì MSB=1 tức là số này hiện tại rất lớn và lớn hơn 10000 nên nó nhận diện được lỗi và yêu cầu nhập lại

Bước 2.6 – Quan sát bộ nhớ. Mở Tools → tích **Labels**, quan sát cửa sổ Labels và tìm địa chỉ nhãn của mảng đơn giá trong Data Segment. Đọc phần tử thứ 3 (chỉ số 2) ở dạng hex và thập phân.

QUAN SÁT Đối chiếu địa chỉ phần tử với địa chỉ nhãn và với lệnh nạp dữ liệu tương ứng trong chương trình.

Bảng 12. Mảng đơn giá trong Data Segment

Địa chỉ nhãn mảng	Địa chỉ phần tử thứ 3	Giá trị (hex)	Giá trị (thập phân)
0x10010070	0x10010078	0x00000852	2130

Câu 2.3 (Áp dụng). Tính tổng tiền lớn nhất mà chương trình có thể in ra với miền nhập hợp lệ và cho biết phép cộng có thể gây tràn có dấu 32 bit hay không. Nếu miền nhập mở rộng đến N kWh thì N lớn nhất là bao nhiêu để tổng tiền vẫn không tràn? Nêu cách tính.

Trả lời:Tổng tiền lớn nhất với miền hợp lệ là x = 9999 , tổng tiền 31489820 dong . Phép cộng không có khả năng gây tràn số 32 bit vì giới hạn của số nguyên có dấu 32-bit là $2^{31} - 1 = 2{,}147{,}483{,}647$ đồng.
*Giới hạn N lớn nhất: 965,000 + (N - 400) *3180 <=2,147,483,647*
➔ *N =675,405 kWh (xấp xỉ)*

Câu 2.4 (Đánh giá). So sánh thiết kế duyệt mảng của Sinh viên với cách viết sáu nhánh `if` riêng cho từng bậc: nêu ít nhất hai tiêu chí (ví dụ kích thước mã, khi biểu giá đổi, khi thêm bậc) và kết luận phương án nào hợp lý hơn cho thiết bị nhúng trong tình huống này.

Trả lời:
Tiêu chí 1: Kích thước mã : sử dụng một vòng lặp duy nhất thì tốn khoảng độ 10 vòng lệnh máy. Còn nếu viết 6 nhánh if riêng biệt, kích thước sẽ tăng rất nhiều do phải lặp lại các cấu trúc so sánh, nhân, cộng và rẽ nhánh 6 lần.
Tiêu chí 2: Khả năng mở rộng và bảo trì: Khi biểu giá điện thay đổi hoặc bổ sung thêm nhiều bậc mới (ví dụ từ 6 bậc lên 10 bậc), phương án duyệt mảng chỉ cần cập nhật dữ liệu trong mảng .data mà không cần chỉnh sửa hay biên dịch lại bất kỳ lệnh nào trong đoạn mã thực thi (.text). Với cách viết 6 nhánh if thì buộc phải can thiệp cấu trúc mã máy, sửa đổi các nhãn rẽ nhánh và kiểm thử lại toàn bộ luồng điều khiển.



THÍ NGHIỆM 3 — Phát giai điệu bằng syscall MIDI

Thời lượng gợi ý: 25 phút · File nộp: Lab01_<MSSV>_nhac.asm · Điểm: 2,0

Tình huống

Thiết bị cần phát một giai điệu ngắn để báo hoàn tất. Giai điệu lưu dưới dạng hai mảng (cao độ và độ dài) trong bộ nhớ dữ liệu, chương trình duyệt mảng và gọi syscall MIDI cho từng nốt. Giai điệu là chủ đề *Ode to Joy* của L. van Beethoven (thuộc phạm vi công cộng); cao độ theo số hiệu nốt MIDI (60 là C4).

Bảng 13. Dữ liệu giai điệu (độ dài tính theo đơn vị nhịp, 1 đơn vị = giá trị `150 + 10*b` ms ở Bảng 1)

Nốt	Tên	Cao độ	Độ dài	Nốt	Tên	Cao độ	Độ dài
1	E4	64	2	9	C4	60	2
2	E4	64	2	10	C4	60	2
3	F4	65	2	11	D4	62	2
4	G4	67	2	12	E4	64	2
5	G4	67	2	13	E4	64	3
6	F4	65	2	14	D4	62	1
7	E4	64	2	15	D4	62	4
8	D4	62	2				

Đặc tả chương trình

- Hai mảng `.word` trong `.data` chứa cao độ và độ dài (đơn vị nhịp) của 15 nốt; số nốt cũng là một dữ liệu trong `.data`.
- Với mỗi nốt: cao độ thực phát bằng cao độ trong mảng cộng `a`; thời lượng (ms) bằng độ dài nhân với `150 + 10*b`; nhạc cụ và âm lượng 100 theo tham số cá nhân. In ra một dòng `<cao do> <ms>` rồi phát nốt bằng syscall 33.
- Đo thời gian chạy của cả vòng phát bằng syscall 30 (hiệu của hai lần đọc `$a0`). Cuối chương trình in `Tong du kien (ms): <so>` (chương trình tự cộng dồn thời lượng các nốt) và `Thoi gian do duoc (ms): <so>`.

Bước 3.1 – Dự đoán. Dùng dữ liệu của Sinh viên trước khi viết mã.

DỰ ĐOÁN Tổng thời lượng dự kiến của 15 nốt là bao nhiêu ms? Nếu thay syscall 33 bằng syscall 31 (giữ nguyên tham số), điều gì xảy ra với âm thanh và với thời gian đo được?

Tổng dự kiến (ms)	Dự đoán khi thay 33 bằng 31
6400	Các nốt phát một lượt một và thời gian sẽ rất bé

Bước 3.2 – Lập trình và quan sát. Viết `Lab01_<MSSV>_nhac.asm` theo đặc tả, chạy và ghi số liệu vào Bảng 14. Nếu máy không phát được âm thanh, vẫn làm đủ các bước: ghi "Không nghe được" ở cột Nghe thấy và dựa vào các dòng in ra và thời gian đo được.

QUAN SÁT Nghe giai điệu có đúng nhịp không? Đối chiếu dòng in cuối với dự đoán ở Bước 3.1.

Bước 3.3 – So sánh ba cách phát. Thử lần lượt ba cấu hình và điền Bảng 14: (A) syscall 33; (B) syscall 31 thay cho 33; (C) syscall 31 rồi ngay sau đó syscall 32 với `$a0` bằng độ dài nốt (ms).

Bảng 14. Ba cấu hình phát nốt

Cấu hình	Nghe thấy	Thời gian đo được (ms)	Số dòng in ra	Nhận xét ngắn
A: 33	Nghe rõ tất cả nốt phát nối tiếp nhau	8100-8300	17	Syscall 33 phát xong nốt rồi mới thực thi lệnh tiếp theo.
B: 31	Các nốt phát cùng lúc nên âm thanh khá hỗn loạn	3	17	Syscall 31 khiến các vòng lặp chạy qua cùng một lúc
C: 31 + 32	Nghe rõ từng nốt nhạc nối tiếp nhau, giai điệu bình thường y hệt như cấu hình A.	6415	17	Syscall 32 đã tạo ra độ trễ khớp với nốt nhạc, kết quả tạo ra giống syscall 33 nhưng thời gian do người dùng kiểm soát

Câu 3.1 (Phân tích). Giải thích chênh lệch (nếu có) giữa tổng dự kiến và thời gian đo được ở cấu hình A. Liệt kê các nguồn gây chênh lệch mà Sinh viên nhận ra.

Trả lời: ở cấu hình A sau khi thực hiện xong một nốt thì còn gặp đỗ trệ của CPU do thực hiện các lệnh khác như lệnh nhảy , lệnh tính toán . Bên cạnh đó còn là chi phí thời gian của hệ điều hành khi xử lí các ngắt I/O liên tục

Câu 3.2 (Áp dụng). Nếu mảng cao độ khai báo bằng `.byte` thay vì `.word`, lệnh nạp phần tử và cách tính địa chỉ phần tử thứ i thay đổi thế nào? Chọn `lb` hay `lbu` cho trường hợp cao độ có thể lên đến 127 và cho trường hợp tổng quát 0–255? Vì sao?

Trả lời: Lệnh nạp phần tử sẽ chuyển từ lw sang lb / lbu . còn i sẽ là addiu $t0,$t0,1 (tiến tới 1 byte thay vì 4 byte)
- *Nên chọn lên lbu vì dải giá trị cao độ MIDI luôn là số không âm do đó dùng lbu sẽ ngăn CPU tự mở ộng bit dấu làm thay đổi sai lệch mã phị phân khi nạp*

Câu 3.3 (Đánh giá). Dựa trên Bảng 14, thiết bị vừa phải phát nhạc vừa phải đọc cảm biến thì nên dùng cấu hình nào? Nêu một rủi ro khi dùng cấu hình B mà không chờ.

Trả lời: Nên dùng cấu hình B , vì syscall 31 trả quyền điều khiển lại cho chương trình ngay lúc đó , giúp giải phóng CPU để tranh thủ đọc dữ liệu khác .
Rủi ro : phát nốt liên tục , không độ trễ , dẫn đến âm thanh hỗn loạn

THÍ NGHIỆM 4 — Đo số lệnh và giải mã lệnh trên chương trình của mình

Thời lượng gợi ý: 30 phút · Dùng chương trình `Lab01_<MSSV>_dien.asm` đã hoàn thành · Điểm: 2,0

> **Tình huống**
> Trước khi triển khai lên vi xử lý nhúng có tài nguyên hạn chế, nhóm phát triển cần ước lượng chi phí của chương trình theo dữ liệu vào và hiểu rõ mã máy sinh ra. Sinh viên đo số lệnh thực thi của chương trình tính tiền điện, xây dựng mô hình chi phí đơn giản và giải mã một số lệnh của chính chương trình mình.

Bước 4.1 – Đo số lệnh thực thi. Mở Instruction C ounter (Connect to MIPS). Với mỗi x ở Bảng 15, đặt lại bộ đếm, chạy, ghi tổng số lệnh thực thi. Đo x = 10 và x = 120 trước.

DỰ ĐOÁN Từ hai số đo đầu tiên, đề xuất công thức N(x) và điền cột Dự đoán cho các hàng còn lại; sau đó đo các hàng còn lại và so sánh.

Bảng 15. Số lệnh thực thi theo số kWh nhập

x (kWh)	Số bậc đã dùng hết	N dự đoán	N đo được	Lệch
10	0	—	36	—
120	2	—	58	—
50	1	36	36	0
250	4	69	69	0
401	5	91	91	0

Câu 4.1 (Phân tích). Viết công thức N(x) của chương trình Sinh viên và giải thích ý nghĩa từng hệ số bằng cách đếm số lệnh trong phần chuẩn bị, trong thân vòng lặp mỗi bậc và trong phần in kết quả. Vì sao N không đổi với mọi x thuộc cùng một bậc?

*Trả lời: N(x) =Ingoai + k*Itrong = 25 + k*11*
Trong đó : Ingoai là số lệnh cố định bên ngoài vòng lặp
Itrong là số lệnh thực thi trong 1 vòng lặp
k: số bậc điện mà x chạm tới
N không đổi vì số lần lặp k chỉ phụ thuộc vào việc x rơi vào ranh giới của bậc nào, không phụ thuộc vào giá trị cụ thể của x lớn hay nhỏ bên trong khoảng ranh giới đó

Câu 4.2 (Đánh giá). Nếu biểu giá có 60 bậc và x nằm ở bậc cuối, N xấp xỉ bao nhiêu? Đề xuất **một** thay đổi nhằm giảm số lệnh mỗi vòng, thực hiện và đo lại ở x = 401; báo cáo N trước, N sau và phần trăm giảm. Nếu không giảm được, lập luận vì sao thân vòng đã tối thiểu.

*Trả lời: N = 25 + 60*11 = 685 lệnh .*
Đề xuất thay đổi : Xóa dòng blez $t4, outloop vì ble $t4, $t1, bac_cuoi đã bao hàm luôn trường hợp này.
Báo cáo : N trước – 91 , N sau – 85 , giảm 6,6%

Bước 4.2 – Giải mã lệnh của chương trình mình. Trong Text Segment của chương trình tiền điện, chọn ba lệnh: một lệnh nhánh trong vòng lặp, một lệnh nạp từ mảng và một lệnh so sánh. Với mỗi lệnh, ghi địa chỉ, mã hex, tách các trường và **tự đọc ra lệnh trước khi nhìn cột Basic**.

QUAN SÁT So khớp kết quả giải mã với cột Basic. Nếu khác, tìm trường nào bị đọc sai.

Bảng 16. Giải mã ba lệnh của chương trình tiền điện

Địa chỉ	Mã hex	Dạng lệnh	Các trường (tên = giá trị thập phân)	Lệnh đọc ra
0x00400034	0x1100fff3	I type	opcode = 4, rs ($t0) = 8, rt ($zero) = 0, immediate = -13	beq $8,$0,0xfffffff3
0x00400050	0x8e490000	I type	opcode = 35, rs = 18, rt = 9, imm = 0	lw $9,0x00000000($18)
0x00400030	0x2a082710	I type	opcode = 10, rs = 16, rt = 8, imm = 10000	slti $8,$16,0x00002710

Câu 4.3 (Áp dụng). Với lệnh nhánh đã chọn, tính (địa chỉ đích − (địa chỉ lệnh + 4)) / 4 và so với trường immediate trong mã hex. Với trường immediate 16 bit, một lệnh nhánh đến được các địa chỉ trong khoảng bao nhiêu byte so với lệnh kế tiếp?

Trả lời: giả sử địch chỉ đích là nhãn wronginput
Ta có : (địa chỉ đích – (địa chỉ lệnh +4)/4) = (0x00400004 – (0x00400034+4))/4 =-13 . Kết quả này khớp với trường immediate của bảng 16.
Trường immediate 16 bit biểu diễn được các số nguyên có dấu từ -2^15 đến 2^15 – 1 (từ -32768 đến 32767). Do mỗi lệnh chiếm 4 byte, một lệnh nhánh có thể nhảy đến các địa chỉ trong khoảng từ -131.072 byte đến +131.068 byte so với lệnh kế tiếp.

Bảng 17. Giả lệnh trong chương trình tiền điện

Giả lệnh trong mã nguồn	Lệnh cơ bản sinh ra	Số lệnh
la $a0, input	lui $1, 0x00001001 ori $4, $1, 0x00000000	2
li $v0, 4	addiu $2, $0, 0x00000004	1
ble $t4, $t1, bac_cuoi	slt $1, $9, $12 beq $1, $0, 0x00000006	2

Câu 4.4 (Đánh giá). Chương trình của Sinh viên có bao nhiêu dòng lệnh nguồn và bao nhiêu lệnh cơ bản trong bộ nhớ? Nếu một giả lệnh nạp địa chỉ nằm trong thân vòng lặp thì nó ảnh hưởng thế nào đến N(x)? Nêu cách bố trí chương trình để tránh chi phí này.

Trả lời:
+ Số dòng lệnh nguồn : 38
+ Số lênh cơ bản : 45
+ Nó sẽ bắt CPU phải thực thi nhiều hơn 1 lệnh cơ bản điều này làm N(x) tăng lên , gây lãng phí tài nguyên máy và giảm hiệu suất chương trình
+Đem các lệnh khởi tạo và lệnh nạp địa chỉ mảng ra bên ngoài vòng lặp

TỔNG KẾT VÀ NỘP BÀI (20 phút)

Danh sách kiểm tra trước khi nộp

☑	Nội dung
☑	File `Lab01_<MSSV>_dien.asm` Assemble không lỗi, có `.data`, `.text`, nhãn `main`, kết thúc bằng syscall 10 và chú thích vai trò thanh ghi.
☑	File `Lab01_<MSSV>_nhac.asm` Assemble không lỗi và in đúng khuôn các dòng yêu cầu.
☑	File báo cáo này đã điền đầy đủ các bảng, câu hỏi và dán ảnh chụp vào các khung nét đứt.
☑	Số liệu trong báo cáo khớp với chương trình nộp (đơn giá, tham số cá nhân và kết quả đo).
☑	Đặt tên file báo cáo: `Lab01_<MSSV>_BaoCao.docx`; nộp ba file theo hướng dẫn của giảng viên.

Tự chấm

Phần	Điểm tối đa	Tự chấm
TN 1	1,5	1,5
TN 2	4,0	4,0
TN 3	2,0	2,0
TN 4	2,0	2,0
Kết	0,5	0,5

Phản tư

Câu R1 (Đánh giá). Từ TN 1 và TN 4, Sinh viên rút ra điều gì về sự khác nhau giữa số dòng nguồn, số lệnh trong bộ nhớ và số lệnh thực thi? Nêu một tình huống cần quan tâm đến từng con số.

Trả lời: số dòng nguồn là số dòng code do người lập trình viết ra (bao gồm cả lệnh giả). Số lệnh trong bộ nhớ là số lệnh máy cơ bản thực sự được nạp vào bộ nhớ . Còn số lệnh thực thi là số lệnh thực sự được chạy bởi CPU
Các tình huống cần quan tâm :
+ Đối với số dòng nguồn : quan tâm khi cần đánh giá khối lượng công việc, tính toán số dòng code hoạc tối ưu mã nguồn,...
+Số lệnh trong bộ nhớ : quan tâm khi thiết kế các hệ thống nhúng , vi điều khiển có bộ nhớ hẹp, nên cần tối ưu để code nhẹ nhất có thể,..
+ Số lệnh thực thi : quan tâm mkhi cần tối ưu hóa điện năng, tăng tốc độ chương trình , tiết kiệm điện , ...

Câu R2 (Phân tích). Từ TN 2, kiểu so sánh có dấu hay không dấu quyết định điều gì trong chương trình? Nêu một tình huống khác ngoài kiểm tra miền nhập mà chọn sai sẽ gây lỗi.

Trả lời: Quyết định các CPU diễn giải bit dấu của thanh ghi 32-bit. So sánh có dấu thì coi MSB = 1 là só âm trong khi so sánh không dấu thì bỏ qua điều đó .
Tình huống : khi thao tác và so sánh các địa chỉ bộ nhớ (Con trỏ). Địa chỉ bộ nhớ MIPS là các số không dấu và có thể vươn tới vùng không gian hệ thống rất cao (bắt đầu bằng bit 1, ví dụ 0x80000000). Nếu dùng so sánh có dấu (slt), địa chỉ cao này sẽ bị hiểu lầm là một "số âm" rất nhỏ, dẫn đến các phép tính ranh giới bộ nhớ, cấp phát động, hoặc so sánh hai con trỏ bị sai bét . Do đó, thao tác với địa chỉ bắt buộc phải dùng lệnh rẽ nhánh/so sánh không dấu

Câu R3 (Đánh giá). Nêu một lỗi hoặc nhầm lẫn Sinh viên đã gặp trong buổi thực hành, Sinh viên phát hiện ra bằng cách nào và bài học rút ra là gì?

Trả lời: Dạ trong TN3 , trong lúc khai báo phần tử cho mảng cao độ và độ dài thì ở mảng độ dài em đã khai báo thiếu một dòng phần tử trong khi mảng cao độ vẫn đủ số note. Điều này khiến 2 mảng không đồng bộ về mặt kích thước Em đã phát hiện ra khi chạy chương trình có một đoạn âm thanh vang lên rất lạ và bị kéo dài , instruction counter cũng sai . Nên khi check lại em đã phát hiện lỗi
Bài học: với ngôn ngữ bậc cao như C++ nếu khai báo sai thì nó sẽ cảnh báo tràn mảng . Nhưng với ngôn ngữ MIPS Assembly thì chương trình sẽ duyệt tới khi hết vòng lặp bất chấp dữ liệu đó là rác hay không . Vì vậy khi lập trình ngôn ngữ máy thì em cần phải kĩ càng trong việc cấp phát, quản lý mảng và các con trỏ.

Tài liệu tham khảo

- Slide bài giảng Chương 2 – Kiến trúc bộ lệnh (Khoa Kỹ thuật Máy tính, UIT).
- MARS 4.5, Help ▸ Help: các mục Instructions, Pseudo-instructions, Syscalls.